

# Muhammad Farooqi — Interview Prep Cheat Sheet

ISARA & AirMatrix — Backend Developer & Computer Vision Intern

---

## ISARA — Backend Software Developer Intern

---

### Q1: What is TLS, SMB, and what does cryptographic risk analysis mean?

---

#### Interviewer question:

*You built cryptographic risk analysis algorithms for TLS, SMB, and plaintext protocols. What is TLS, what is SMB, and what does cryptographic risk analysis actually mean — what were you analyzing and what was the output?*

#### Model Answer:

TLS is the encryption protocol used for secure web traffic — HTTPS runs on TLS. SMB is a file sharing protocol used in Windows networks. Plaintext means unencrypted communication — things like HTTP or FTP where data is sent completely in the open.

Cryptographic risk analysis means scanning a network and identifying which devices are using weak or outdated configurations. A concrete example: a device still using TLS 1.0 or 1.1 instead of TLS 1.3 is a risk — those older versions have known vulnerabilities like POODLE and BEAST attacks. A device using SMB v1 is a critical risk — that's the protocol EternalBlue exploited in the WannaCry ransomware attack.

My algorithm would scan network traffic using Node.js and GraphQL, identify which protocol version each device was using, and output a risk score. Devices on plaintext protocols scored highest risk, devices on outdated TLS versions scored medium, devices on current TLS 1.3 scored low. The aggregated score gave a health picture of the whole network.

■ Two examples to always remember: TLS 1.0/1.1 = POODLE/BEAST attacks. SMB v1 = WannaCry/EternalBlue = critical risk.

### Q2: You resolved 355+ failures during regression testing — what does that mean?

---

#### Interviewer question:

*Your resume says you collaborated with QA to resolve 355+ failures during regression testing, improving product health by 88.75%. What does regression testing mean in this context and what were those failures?*

#### Model Answer:

Regression testing means running the full test suite against a new version of the product to make sure existing functionality wasn't broken by new changes. At ISARA the product was a network security analysis tool, so regression failures meant the risk scoring algorithms were producing wrong outputs — devices being scored incorrectly, protocols being misidentified, or edge cases causing crashes.

355+ failures sounds like a lot but in a large test suite that's not unusual when significant new features land. My job was to work through those failures with QA, identify which were genuine bugs vs flaky tests, fix the bugs on the backend, and get the suite back to green. 88.75% improvement meant going from a large number of failing tests down to a small residual that were either known issues or intentionally deferred.

■ *If pressed on 88.75%: that's (355 - remaining failures) / 355. Don't get caught up in the exact math — the point is you worked systematically through a large backlog with QA.*

---

## AirMatrix — Computer Vision Intern

---

### Q1: What is IoU and what does 95% accuracy at 0.5 IoU mean?

---

#### Interviewer question:

*You trained a YOLOv8 model achieving 95% accuracy at 0.5 IoU. What is IoU, what does 0.5 IoU mean specifically, and why does the threshold matter?*

#### Model Answer:

IoU stands for Intersection over Union. It measures how much your predicted bounding box overlaps with the ground truth bounding box. You take the area where the two boxes overlap — the intersection — and divide it by the total area covered by both boxes combined — the union. A perfect prediction gives IoU of 1.0, no overlap gives 0.

0.5 IoU means we consider a detection correct if the predicted box overlaps the ground truth box by at least 50%. So if I predict a car is here and the actual car is there, as long as my box covers at least half the real box, it counts as a correct detection.

Why the threshold matters: a loose threshold like 0.5 is forgiving — it accepts detections that are roughly in the right place. Autonomous vehicles typically use stricter thresholds like 0.75 or 0.9 because you need precise localization. At Kodiak, knowing where a car is within 50% isn't good enough — you need to know exactly where it is to plan safely around it.

■ *One sentence to memorize: IoU is intersection over union — how much your predicted bounding box overlaps the ground truth, expressed as a ratio from 0 to 1.*

### Q2: Why does annotation quality matter — give a concrete example of bad annotation?

---

#### Interviewer question:

*You annotated 3,000+ traffic images. Why does annotation quality matter so much, and what specifically makes a badly annotated bounding box dangerous for a model like YOLOv8?*

#### Model Answer:

Garbage in, garbage out — but specifically: if I annotate a car but my bounding box is too loose and includes part of the sidewalk and a pedestrian next to the car, the model learns that 'car' includes that extra context. At inference time it starts drawing boxes that are consistently oversized, eating into adjacent objects. That makes it harder to detect pedestrians standing near cars because their pixels get absorbed into the car's predicted box.

Another example: inconsistent annotation — sometimes I label a partially occluded car, sometimes I skip it. The model sees the same type of object labeled differently and gets confused about when to fire a detection. At inference time this shows up as missed detections on partially visible vehicles — exactly the scenario that matters most on a highway where cars are constantly partially occluded.

This is why annotation quality directly drove the 95% accuracy — tight, consistent bounding boxes meant the model learned clean boundaries. Roboflow has built-in tools to flag inconsistent annotations and review quality before training.

### Q3: Speed estimation — how do you get from 2D pixels to km/h?

---

#### Interviewer question:

*You built vehicle speed estimation. A camera gives you 2D pixel coordinates — how do you actually get from pixel positions to a real-world speed in km/h?*

#### Model Answer:

A camera only gives you 2D pixels so you need a real-world reference to convert pixels to meters. We used the known width of a standard Canadian road lane — about 3.7 meters — as our spatial calibration reference. If a lane spans  $X$  pixels wide in the image, then  $1 \text{ pixel} = 3.7/X \text{ meters}$ . That gives you your pixel-to-meter scale factor for the whole image.

For speed: track the vehicle's bounding box center across consecutive frames, measure how many pixels it moves along the direction of travel between frames, multiply by your pixel-to-meter factor to get meters traveled, divide by time between frames to get meters per second, then convert to km/h.

The limitation is perspective distortion — this works cleanly for vehicles moving parallel to the camera plane. If the camera angle changes or the car is moving toward the camera, the pixel displacement doesn't map cleanly to real-world distance. For a fixed traffic camera that's acceptable. For a moving platform like an autonomous truck, you'd need full 3D localization from LiDAR or stereo cameras to do it properly.

■ *The limitation at the end is what separates a good answer from a great one. Kodiak will specifically care about this — their trucks are moving platforms, not fixed cameras.*

### Q4: What is YOLOv8 and why use it over other detection models?

---

#### Interviewer question:

*Why did you use YOLOv8 specifically for vehicle detection and not another model?*

#### Model Answer:

YOLO stands for You Only Look Once — it's a single-stage object detector, meaning it predicts bounding boxes and class labels in one forward pass through the network. This makes it very fast compared to two-stage detectors like Faster R-CNN which first propose regions then classify them.

YOLOv8 specifically was the latest version at the time with the best accuracy-speed tradeoff for real-time detection. For a traffic camera use case where you need to process video at 30 frames per second, inference speed matters as much as accuracy. YOLOv8 also has a clean Python API and integrates directly with Roboflow datasets, which made the training pipeline straightforward.

The tradeoff: YOLO is fast but less accurate on very small objects compared to two-stage detectors. For vehicles in traffic camera footage that's acceptable — cars are large enough in frame. For detecting small objects at long range like Kodiak needs, you'd want a more specialized architecture.

■ *If asked about YOLOv11 on your resume — it's the newer version you've used at WATonomous. Same YOLO family, newer architecture with improved performance.*